

Agent Orchestration

Orchestrating Multiple Agent Sessions

Day 1 - Session 1, Topic 3

What We'll Cover

1. Why multiple sessions create a coordination problem
2. Parallel sessions and scheduled Claude Code routines
3. Handoffs and integration gates
4. Demos: Worktrees and Handoff Gate
5. Lab 3.1: Parallel Agent Coordination

What Is an Agent Session?

An agent session is one running interaction with its own:

- Conversation and working context
- Assigned goal and instructions
- Available tools and permissions
- View of repository and workspace state

Two sessions do not automatically share context, decisions, or uncommitted edits.

Why Use Multiple Sessions?

Separate sessions can work concurrently when tasks have stable boundaries.

- Reduce elapsed time for independent work
- Keep each session focused on a bounded task
- Apply different models, tools, or permissions where needed

Condition: The tasks must have low coupling and explicit integration points.

The Problem We Are Solving

Two agents implement email and Short Message Service (SMS) notifications at the same time.

- Both edit `router.py`, creating overwrite or merge risk.
- Each makes different assumptions about the shared schema.
- Focused tests pass, but the combined behavior fails.
- Results remain trapped in separate chat histories.

Need: Isolate work, transfer evidence, merge in dependency order, and test the combined state.

What Is a Task Card?

A task card is a written assignment for one bounded unit of work—small enough to delegate and clear enough to verify.

- **Outcome:** named deliverable and acceptance check
- **Boundary:** context, allowed files, and non-goals
- **Coordination:** dependencies and handoff evidence

Without a card, sessions infer scope and success; with one, the orchestrator can delegate, sequence, and verify work before integration.

Isolate the Workers

Separate files first, then coordinate evidence

Choose the Coordination Surface

Approach	Coordinator	Best fit
Subagents	Main session	Focused side tasks
Agent view	Human operator	Independent monitored sessions
Agent teams	Lead agent	Experimental collaborative work
Dynamic workflows	Script	Large, repeatable multi-pass work

Use the lightest mechanism that satisfies the coordination need.

What `/schedule` Schedules

A Routine saves a prompt, repositories, environment, connectors, and trigger.

- It runs in the cloud even when your laptop or terminal is off.
- Each trigger starts a Claude Code session that can use committed repository skills; scheduling alone does not define agent ownership or integration.
- Example: `/schedule daily PR review at 9am`.

`/loop` is session-local, needs an active session, and recurring loops expire after seven days.

Authentication and Authorization Must Last

- **Authenticate:** `/schedule` requires a `claude.ai` subscription login; API-key and cloud-provider logins do not expose it.
- **Authorize:** routines use your linked GitHub and connector identities; included connectors may write without per-run prompts.
- **Expiry:** missing or expired GitHub access skips runs for up to 72 hours, then the routine turns off.
- **Control:** limit repositories and connectors, reconnect and re-enable when needed, and require human review before merges or deployments.

Sources: [Claude Code routines](#) · [Scheduled tasks](#)

Worktrees Isolate Filesystem State

Each worktree has its own directory, index, branch, and uncommitted edits.

All worktrees share repository history and Git objects.

Isolation prevents accidental edit visibility across sessions.

It does not prevent incompatible assumptions or semantic conflicts.

What Does a Worktree Protect?

A worker has its own Git worktree. What does that isolation actually protect?

- A) Dependency sequencing
- B) Uncommitted edit visibility
- C) Combined test success
- D) Approval requirements

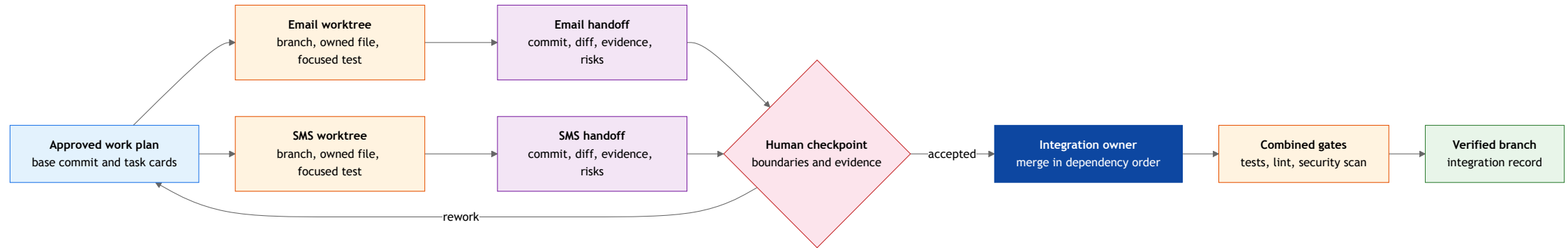
Chat Check: Type your answer (A, B, C, or D) in the chat window now.

Worktree Protection: The Answer

B - Uncommitted edit visibility

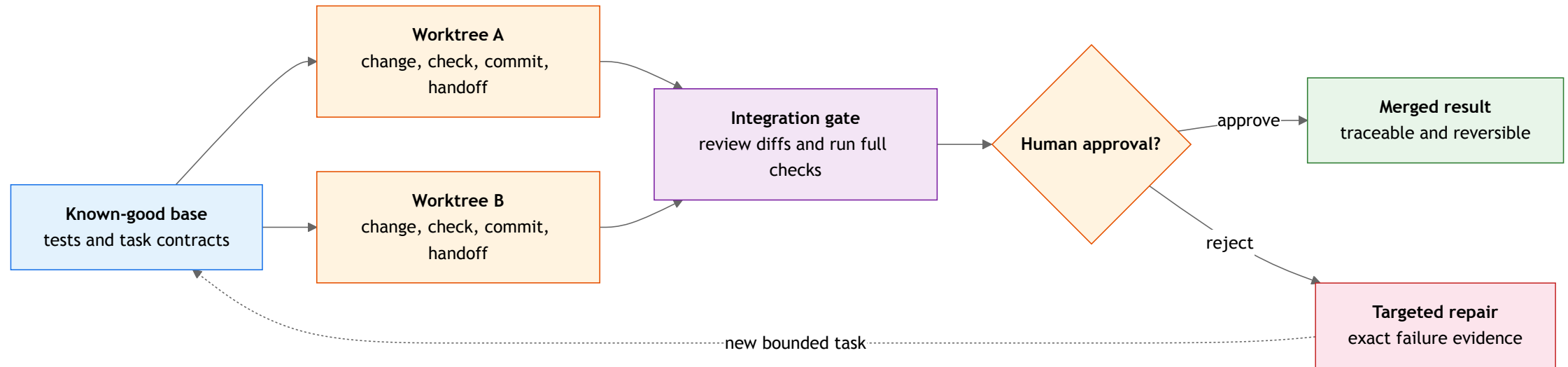
Each worktree isolates its directory, index, branch, and uncommitted edits. It does not prevent semantic conflicts or guarantee integrated behavior.

Parallel Worktree Orchestration



Generic Multi-Agent Worktree Flow

REFERENCE ONLY



Coordinated Specialist Agents (SaaS)

SaaS: Software as a Service

REAL-WORLD SCENARIO

SaaS (GitLab - Duo Agent Platform):

- Specialized agents handle refactoring, security scans, and research asynchronously across the development lifecycle.
- Multi-agent flows can scan a merge request, review code, generate tests, and draft documentation.

Coordinated Specialist Agents (Financial Services)

SKIP

REAL-WORLD SCENARIO

Financial services (Deloitte - Personal Wealth Advisor):

- Six specialist agents cover market data, portfolios, trades, advice, documents, and general assistance.
- The coordinated roles separate concerns while supporting recommendations and financial transactions.

Workspace Assignment Contract

1. Base commit, branch, and worktree path
2. Task-card identifier and dependency state
3. Allowed files and prohibited shared files
4. Focused verification command
5. Integration owner and approval checkpoint

All parallel workers start from the same approved base.

Demo: Git Worktree Isolation

REFERENCE ONLY

From the companion repo, run `python3 day1/concepts/demos-conceptual/demo-git-worktree-isolation.py`.

- Git evidence: two real worktrees and branches are created.
- Isolation: worker files remain absent from `main` before merge.
- Integration: explicit merges create the combined history.

Uses temporary directories, Git, and the Python standard library.

Demo: Two Coordinated Sessions End to End

From the companion repo, run `python3 day1/demos/demo-two-session-orchestration.py`.

- Isolation: two real worktrees; `main` stays at the base commit.
- Handoffs: each records its real commit and real focused-test result.
- Ordered merge: one human approval point before the second merge.
- Conflict: Git reports `UU router.py`; the approved order wins and stays green.

Requires Git; runs in a temporary clone with no API key.

Notebook: `day1/demos-notebook/demo-two-session-orchestration.ipynb`.

Admit Evidence, Not Claims

Handoffs and checks control integration

Durable Handoff Contract

Evidence	Why integration needs it
Base and resulting commits	Reconstruct the exact patch lineage.
Changed files	Confirm ownership boundaries.
Decisions and assumptions	Expose effects on other tasks.
Focused command and result	Verify the bounded outcome.
Risks and next checkpoint	Decide whether work can advance.

Why Record Both Commits?

Why should a durable handoff include the base and resulting commits?

- A) Calculate token cost
- B) Confirm effort level
- C) Prove patch lineage
- D) Avoid all conflicts

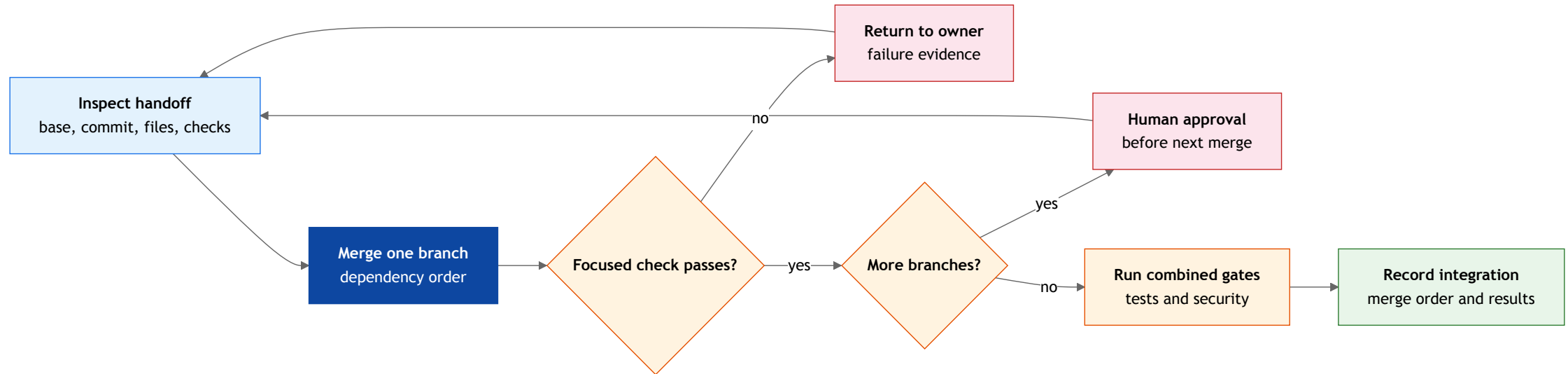
Chat Check: Type your answer (A, B, C, or D) in the chat window now.

Commit Evidence: The Answer

C - Prove patch lineage

The two commits prove the approved starting point and identify the exact resulting patch. They do not prevent merge conflicts.

Integrate in Dependency Order



A Clean Merge Is Not Compatibility

Git detects overlapping text edits.

Tests detect some behavioral incompatibilities.

Security scanning detects a different class of integration risk.

Human review checks architecture, authorization, and business impact.

Use focused checks after each merge and full gates on the combined state.

Why Run Full Gates?

Each branch passed focused tests and merged cleanly. Why test the combined state?

- A) Git requires it
- B) It uses live data
- C) Catch combined-state risks
- D) Agents cannot test

Chat Check: Type your answer (A, B, C, or D) in the chat window now.

Full Gates: The Answer

C - Catch combined-state risks

Focused tests prove bounded outcomes. Full gates catch semantic incompatibilities and security failures that appear only after integration.

Demo: Handoff Integration Gate

REFERENCE ONLY

From the companion repo, run `python3 day1/concepts/demos-conceptual/demo-handoff-integration-gate.py`.

- Base gate: both workers must start from the approved commit.
- Ownership gate: a router edit blocks a channel handoff.
- Risk gate: passing tests do not erase an unresolved concern.

Runs locally without an API key or network access.

Demo: Integration Gate on the Real Diff

REFERENCE ONLY

From the companion repo, run `python3 day1/demos/demo-integration-gate-real-diff.py`.

- Ownership: read from the real `task-cards/*.md` files.
- Changed files: taken from `git diff --name-only`, not declared.
- Both commits pass their focused tests, yet one is blocked.
- Blocked reason: the diff reached `router.py`, outside its card.

The clearest evidence that a green test cannot detect a boundary violation.

Notebook: `day1/demos-notebook/demo-integration-gate-real-diff.ipynb`.

Demo (Optional): Handoff Summary Without a Key

REFERENCE ONLY

From the companion repo, run `python3 day1/demos/demo-handoff-summary.py`.

- Structural fields measured from Git and tests; only the prose is generated.
- A small local model (`flan-t5-base`) summarizes the real diff.
- Model output is rejected if it echoes the diff, then falls back cleanly.
- `summary_source` records which path produced the field.

Optional and skippable. Completes without the model or `transformers`.

Notebook: `day1/demos-notebook/demo-handoff-summary.ipynb`.

Enterprise Translation

Local course control	Enterprise control
Git worktree	Ephemeral cloud workspace
Short-lived branch	Trunk-based pull request
Local acceptance command	Continuous integration required check
Security unit test	Static and dynamic security scanning
Integration owner	Protected branch or merge queue

What Industry Experiments Show

Anthropic's compiler experiment used 16 agents and nearly 2,000 sessions.

Progress depended on tests, task locks, isolated environments, and clean feedback.

Parallelism stopped helping when every agent encountered the same kernel failure.

The harness had to expose independent subsets before workers could specialize.

This is a case study, not a general productivity guarantee.

Lab 3.1: Parallel Agent Coordination

Open `day1/lab3_1_parallel-agent-coordination/` in the companion repo.

Goal: Complete two channel workers in isolated Claude Code sessions.

1. Create worktrees and enforce non-overlapping ownership.
2. Review commits, diffs, checks, and handoffs.
3. Approve, merge in order, run combined gates, and resolve the fixture.

Zero external dependencies; each lab starts from a fresh repository.

Key Takeaways

1. Worktrees isolate edits but do not solve logical coordination.
2. Every workspace needs a common base, exclusive scope, and focused check.
3. Durable handoffs carry commits, diffs, evidence, risks, and checkpoints.
4. Integration follows dependencies and verifies the combined state.
5. Human approval remains essential at security and high-risk boundaries.

Course Wrap-Up

Plan, route, coordinate, and verify

Day 1 Outcomes

You can now:

1. Decompose a feature into bounded, verifiable task cards.
2. Select model and effort from complexity, risk, and cost evidence.
3. Coordinate isolated sessions with explicit ownership and handoffs.
4. Integrate in dependency order with automated and human gates.

Demo: Interactive Multi-Agent Coordination

Open `day1/demos/interactive-multi-agent-coordination.html` from the companion repository.

- Compare Subagents, Agent view, agent teams, and dynamic workflows.
- Identify who coordinates each surface and when it fits.
- Inspect the workspace, handoff, and integration controls that every surface still requires.

Select a coordination surface, then trace its relationship to the shared controls.

Built with Archify from `interactive-multi-agent-coordination.architecture.json`. Self-contained; no server, install, API key, or network required.

Demo: Interactive Parallel Orchestration

Open `day1/demos/interactive-parallel-orchestration.html` from the companion repository.

- Follow isolated workers through evidence handoffs and ordered admission.
- Observe the combined integration gate and targeted repair loop.
- Use the chapter controls and reader-controlled trace to reveal each execution stage.

Compare what may run in parallel with what must remain sequential.

Built with Archify from `interactive-parallel-orchestration.workflow.json`. Self-contained; no server, install, API key, or network required.

Put It to Work

Complete this privately:

When [specific workplace situation] occurs, I will [orchestration practice], using [artifact or evidence], and I will know it worked when [observable result].

Exchange plans with one partner. Ask:

1. Which boundary, dependency, or approval condition is explicit?
2. Which check proves the result?

Revise one line. Keep a private note instead if you prefer.

Questions?

Keep the plan and evidence durable after the sessions close.